

Infix to Postfix Converter

Expression Evaluator

Name	Haris
Course	Compiler Construction LAB
Date	10-06-2026
Registration Number	2023f-mulbcs-157

Abstract

This report describes a C++ program that converts infix mathematical expressions into Reverse Polish Notation (postfix) and evaluates the result. The program handles multi-digit numbers, the standard operators (+, -, *, /, ^), nested parentheses, and error conditions such as mismatched brackets and division by zero. The implementation uses Dijkstra's Shunting Yard algorithm and relies only on the C++11 standard library.

1. Introduction

Infix notation is how people normally write mathematical expressions, such as $3 + 5 * 2$. Evaluating this format directly requires handling operator precedence and parentheses. Postfix notation solves this by placing each operator after its operands, making the expression straightforward to evaluate from left to right without any special rules.

This program takes an infix expression from the user, converts it to postfix, and computes the result. It is written in standard C++11 with no external dependencies.

2. System Architecture

The program processes input through three stages in sequence:

Stage	Component	Description
1	Tokenizer	Reads the input and splits it into tokens (numbers, operators, parentheses).
2	Shunting Yard	Reorders the tokens from infix into postfix using a stack and an output queue.
3	Evaluator	Processes the postfix queue and returns the final numerical result.

2.1 Data Structures

- **Operator Stack:** A last-in, first-out structure that temporarily holds operators while their correct position in the output is being determined.
- **Output Queue:** An ordered list that holds tokens in postfix order, used later for evaluation.

3. Algorithm

The Shunting Yard algorithm reads tokens one at a time and applies the following rules:

- **Number token:** Send it directly to the output queue.
- **Operator token:** Move any operators on the stack with higher or equal precedence to the output queue first, then push the new operator onto the stack.
- **Left parenthesis '(':** Push it onto the stack as a scope marker.
- **Right parenthesis ')':** Move operators to the output queue until the matching '(' is found, then discard both parentheses.
- **End of input:** Move all remaining operators from the stack to the output queue. If any parentheses remain on the stack, raise a mismatched-bracket error.

4. Modules

Each function has a single, well-defined responsibility:

Module	Responsibility
struct Token	Holds a token's type and its string value.
precedence()	Returns an integer priority for each operator (+/- = 1, */÷ = 2, ^ = 3).
isRightAssociative()	Returns true for ^ so that 2^3^2 evaluates right-to-left.
tokenize()	Reads the raw input string and returns a list of Token objects.
infixToPostfix()	Applies the Shunting Yard algorithm and returns the postfix token list.
evaluatePostfix()	Evaluates the postfix list using a number stack and returns a double result.

4.1 Operator Precedence

Operator	Symbol	Precedence	Associativity
Addition / Subtraction	+, −	1 (Lowest)	Left
Multiplication / Division	*, /	2 (Medium)	Left
Exponentiation	^	3 (Highest)	Right

5. Error Handling

Errors are caught using C++ try-catch blocks with `std::runtime_error`. When an error occurs, execution stops and a message is printed.

Error	Trigger	Response
Mismatched Parentheses	An opening or closing bracket has no matching pair.	Throws runtime_error; stops conversion.
Division by Zero	A denominator evaluates to zero.	Throws runtime_error; stops evaluation.
Invalid Character	An unrecognised symbol is found in the input.	Throws runtime_error during tokenization.

6. Test Cases

6.1 Valid Expressions

Infix Input	Postfix Output	Result
3 + 5	3 5 +	8
10 - 3 * 2	10 3 2 * -	4
3 + 5 * (2 - 1)	3 5 2 1 - * +	8

6.2 Error Cases

Input	Error Message
(3 + 5 * 2	Error: Mismatched parentheses.
10 / 0	Error: Division by zero encountered.

7. Build Instructions

- **Compiler:** C++11 or later (g++, clang++, MSVC)
- **Dependencies:** Standard C++ Library only

To compile the program, run:

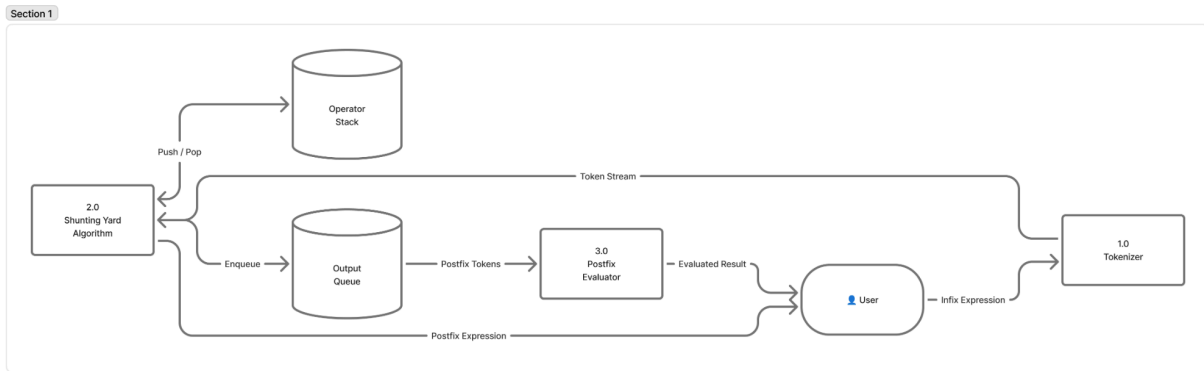
```
g++ -o converter infix_to_postfix.cpp -std=c++11
```

8. Conclusion

This program provides a clean, modular implementation of infix-to-postfix conversion and expression evaluation in C++. Separating the logic into tokenization, conversion, and evaluation stages keeps the code readable and easy to extend.

The Shunting Yard algorithm correctly handles all standard precedence and associativity rules, while the exception-handling layer ensures the program responds gracefully to invalid input. This design serves as a solid starting point for more advanced compiler front-end components.

9. DataFlow Diagram



WorkFlow Diagram

